NAWEDDERBURN
WEB DEVELOPER

TECHNICAL IMPLEMENTATION

LANDING PAGE SLIDESHOW

PREPARED BY

N. A. Wedderburn

PUBLISHED

March 1, 2026

VERSION

1.1

CONTENTS

PROJECT OVERVIEW 5

Halt Animations While Page Loads 5
Priority Loading Above the Fold 5
Component Dimensions Determined by Parent Container 5
Scroll Indicators for Scrollable Containers 5
Responsive & Configurable Slide Display 6
Responsive, Accessible Slide Navigation 6
Slide Presentation Configuration 6
Print with Screen Display or Minimal Formatting 7
Color Reversal for Dark Mode 7

PROJECT IMPLEMENTATION 8

Body 8
Controlling Print Format 8
Color Palette 11
Raster Image Sizing 11
Word Break for Narrow Containers 12
Maximum Image Width 12
Anchors 12
Paged Media 14
Component – Scrollable Container 14
Rationale 14
Implementation 15
Implementation Caveats & Limitations 17
Unsuccessful Implementations 19
Considered Implementations 19
Component – Loader 20
Rationale 20

©N. A. WEDDERBURN

Render & Parser Blocking JavaScript Module 20
Hide Page Content & Pause Animations 20
Construct & Display the Loader 21
Remove the Loader 21
Preloading Critical Assets 21
Animations 22
Algorithm 25
Error Handling 25
Unsuccessful Implementations 26
Component – Product 27
Container Query Implementation 27
Stylized Grid Layout 27
Stylized Display 28
Component – Slide 28
Structure 28
Decorative Header & Background Images 29
Navigation 31
Margins & Padding 33
Content Images 34
Stylized Display 34
Layout – Slideshow Presentation 34
Fixed Dimension Slide Layout 34
Margin Overflow 35
Scroll Snapping 35
Stylized Display 35
Layout – Slide Preview Presentation 36

©N. A. WEDDERBURN

Fixed Dimension Slide Layout 36

Grid Structure 36

Stylized Display 37

Layout – Full Document Slideshow Presentation 37

Header – Main 38

Footer – Main 38

Slides 39

Presentation – Indulgent Hydration 39

Layout – Intro Slide 39

Layout – New Arrivals Slide 41

Layout – Collection Information Slide 43

APPENDIX **46**

References 46

Figures & Tables 47

©N. A. WEDDERBURN

PROJECT OVERVIEW

Landing page with multi-slide presentation layout for fictional company “Virna & Nuris”. Responsive components implemented with container queries that adjust to slideshow presentations in flow layouts, fixed dimension layouts, and preview thumbnails.

Halt Animations While Page Loads

A loader animation determined by the user’s motion preference is implemented to prevent the intro slide animations from executing before the page loads. If JavaScript is enabled, a render and parser blocking script inserts an animated page loader into the DOM. Page elements other than the loader are hidden until the page loads or a maximum loader display time is exceeded. If the loader fails at any point, recovery error handlers remove the loader if it was created and the body elements are displayed.

Priority Loading Above the Fold

Give load priority to critical external assets on the first slide. Each critical asset is linked with a high fetchpriority in the DOM header.

Component Dimensions Determined by Parent Container

Container queries are used to configure component properties based on the dimensions of the parent container instead of the viewport. Properties can be adjusted based on the width of the parent or both width and height in a fixed dimension layout.

Fixed dimension containers are used to implement slide components that span the full viewport dimensions for a slideshow presentation. The slides can also be drastically reduced in size and displayed in a slideshow preview presentation.

Scroll Indicators for Scrollable Containers

©N. A. WEDDERBURN

Scroll indicators are implemented for devices with auto-hide scrollbars to indicate overflowing, scrollable content. The “scroll shadows” design pattern is used to form a composite background image using gradients to show and hide scroll indicator images.

Responsive & Configurable Slide Display

A slide is a configurable component. Default properties can be overridden for each slide implementation and unique layouts can be applied. The structure is implemented with container queries so its dimensions are determined by the dimensions of its container.

A slide is displayed in a flow layout by default. When slides are displayed in a slideshow with adequate viewport and container dimensions, the slides are displayed in a fixed dimension layout. With fixed slide dimensions, content overflow is controlled by clipping or scrolling.

Responsive, Accessible Slide Navigation

The display of the keyboard and screen reader accessible navigation is based on the slide display. For flow layout, the navigation container is sticky positioned, keeping the navigation in the vertical center of the viewport until it reaches the bottom of the slide container. For fixed dimension layout, slide navigation anchors are absolutely positioned on the bottom corners of the slide.

Slide Presentation Configuration

Slides can be displayed in a slideshow presentation or a slide preview presentation.

In a slideshow presentation, slides are displayed in a flow layout or fixed dimension layout based on the full dimensions of the viewport. If the presentation encompasses the full document, the page header and footer are incorporated into the slideshow display. Scroll snapping is implemented on the root element of pages that contain slide show presentations to allow slides to snap into place while scrolling.

In a slide preview presentation, slides are displayed as preview thumbnails. The presentation is configured as a grid with fixed dimension slides as the grid items. The responsive dimensions of the grid items can be defined for each presentation using custom properties for slide width and height.

©N. A. WEDDERBURN

Print with Screen Display or Minimal Formatting

Components can be printed unformatted or with the stylized formatting usually applicable only for screen display. This allows for printing slides as they appear on screen in the presentation preview layout.

Color Reversal for Dark Mode

All surfaces have light background colors and contrasting dark text. When user's preferred color scheme is dark, the colors are reversed such that the surfaces have dark background colors and the text is lighter in contrast.

©N. A. WEDDERBURN

PROJECT IMPLEMENTATION

Body

Controlling Print Format

Rationale

Print media queries are usually used to simplify a web page for the least of amount of paper and ink usage. However, in this project, components can be printed with the stylized formatting usually meant for screen. Slides should be printed without stylized display, except when slides are in the preview layout. If a slide is printed with stylized display, the components within the slide must also be printed with stylized display.

Since slides can be printed with screen formatting, there is no distinction between screen and print styles. In order to print without screen formatting, an indicator is required to toggle between formatted versus unformatted styling.

The space toggle trick was deemed a viable design pattern for this implementation even though it may remove all styles from an element, including the default user agent styles. Removing default styling just for print is acceptable since unformatted print does not affect critical application functionality.

Implementation

The space toggle trick is used to toggle formatting styles using a single custom property, "--stylized-display". The value of the custom property is determined by the "print-unformatted" class added to elements in the DOM.

DOM IMPLEMENTATION

To indicate that a component should not have stylized display for print, the class "print-unformatted" is applied to the component, or to an ancestor of the component.

CSS IMPLEMENTATION

Screen & Formatted Print

In the root, the custom property "--stylized-display" is set to "initial":

```
--stylized-display: initial;
```

The value “initial” indicates stylized/formatted display is turned on.

MODIFY PROPERTY VALUE

The keyword “initial” is an invalid value. When this invalid value is used in a var function configured with a default value parameter, the default value is used to modify the property’s value.

For example, the following padding property will be changed to 1em:

```
padding: var(--stylized-display, 1em);
```

The “--stylized-display” value of “initial” is invalid, so the default value parameter to the var function, 1em, is used to set the padding property.

PRESERVE PROPERTY VALUE

The keyword “initial” is an invalid value. When this invalid value precedes a property value, the entire rule is invalid. No changes are made to the property and the current property value in the cascade hierarchy persists.

For example, the following padding property will not be altered:

```
padding: var(--stylized-display)1em;
```

The “--stylized-display” value of “initial” is invalid, making the entire rule invalid. No changes will be made to the padding property, persisting the padding property’s current value.

Unformatted Print

In the root, for printed elements defined with the “print-unformatted” class, the custom property “--stylized-display” is set to “ ” (space character):

```
@media print {
  .print-unformatted,
  .print-unformatted * {
    --stylized-display: ;
  }
}
```

The value “ ” (space character) indicates stylized/formatted display is turned off.

MODIFY PROPERTY VALUE

The keyword “ ” (space character) is a valid value. When this valid empty value precedes a property value, the empty value is ignored and the rule is executed.

For example, the following padding property will be changed to 1em:

```
padding: var(--stylized-display)1em;
```

The "--stylized-display" value of " " is valid, making the rule valid. The space character is ignored as whitespace, so the value 1em is used to set the padding property.

SET PROPERTY VALUE TO DEFAULT

The keyword " " (space character) is a valid value. When this valid empty value is used in a var function configured with a default value parameter, the property's value is reset to its generic default value.

For example, the following padding property will be set to 0px.

```
padding: var(--stylized-display, 1em);
```

The "--stylized-display" value of " " is valid so the padding property is set to its generic default value.

Note that setting a property value to its generic default value creates a complete CSS reset and cascade loss.

Known Issues

COMPLETE CSS RESET

When the space toggle trick sets a CSS property using the var function and the space character, it does not unset/revert the property value. The trick sets the property value to the default value for a generic element. The default value **is not** the default value determined by the user agent style sheet for the specified element. The default value is the base value for an element. For example, the default display on a div element is "block" as determined by most user agent style sheets. The space toggle trick will set the display to the generic default display value "inline".

CASCADE LOSS

An issue with complete CSS reset is the loss of the cascade. When the property values are set to their base value, they cannot be unset or reverted to a previous value. Normally, when the value of a property is set to an invalid value, the value is unset or reverted to a previous version in the cascade order. The previous version would likely be the default user agent stylesheet value. However, an invalid value returned from the var function sets the property value to the default value instead of unsetting or reverting to a previous version, causing cascade loss.

INCONSISTENT PRINT DISPLAY

Occasionally, slide background images are not displayed. Slide content may also be excluded from the display, especially in portrait orientation. These issues appear to be based on page margins and scale preventing content display. Firefox has more print margin options and sometimes a combination of

©N. A. WEDDERBURN

reduced margin and scale can allow for proper display, but it is inconsistent. Sometimes multiple print attempts allow for proper display, but is also inconsistent.

Containers with scrolling content may also be excluded from display. If the container content is larger than the container dimensions creating overflow, the entire container may be excluded instead of just clipped. This issue is user agent dependent and is more frequent in Firefox.

It is not possible to use developer tools to troubleshoot these print issues. The content displayed using the print emulator uses screen display functions, not the actual print engine.

Color Palette

The colors are defined using Hue Saturation Lightness (HSL). The color palette consists of 2 branded colors (primary, secondary) and a gray neutral (tertiary). The colors each have 5 contrast levels in relation to the background color: highest, high, medium, low, and lowest. The highest contrast color has the color's defined lightness property. The remaining 4 colors are calculated directly in the CSS by gradually increasing the color's lightness property using a lightness factor and lowest lightness percentage. See the A scalable naming convention for style-variables article that describes creating a color palette based on contrast to the background color.

Dark Mode

The page is designed with all surfaces having light background colors and contrasting dark text. The colors are reversed within a prefers-dark-mode media query such that the surfaces have dark background colors and the text is lighter in contrast. See Building a color scheme reference.

Shadows

All drop shadows are the same dark gray color. A shadow intensity custom property is added to the shadow color’s alpha value. For dark mode, the shadow intensity custom property is increased to create a darker shadow for more contrast with the background. Note that the page background color is not black in dark mode. The page background is dark gray and the shadow color is a darker gray. See Building a color scheme reference.

Raster Image Sizing

HTML

Raster images that are included in the DOM are downloaded based on Device Pixel Ratio (DPR) using srcset and w-descriptors. Images are prepared with multiple sizes and stored in directories based on size.

©N. A. WEDDERBURN

The images are WEBP format with JPEG or PNG fallbacks.

Note that when defining the responsive size of an image using srcset, container query units cannot be used in the sizes attribute. The sizes must be based on the viewport dimensions, not the container dimensions. Using container units has been proposed, but not implemented by any browsers.

Note that if an image is used multiple times on the same page, but with different size requirements, multiple versions of the same image will be downloaded. This increases the number of requests and load time.

CSS

Raster images used as background images defined in CSS are selected based on viewport width media queries. The images are WEBP format with JPEG or PNG fallbacks.

Only 2 sizes of background images are provided – large and x-large. The background image image-set CSS function uses x-descriptors that define Pixels Per Inch (PPI). If a screen is physically large but has low pixel density, a small image file may be selected that is displayed as distorted or pixelated on a large screen. So, large background images are used regardless of the viewport dimensions.

Word Break for Narrow Containers

For all containers, individual words will break at any point to avoid horizontal scroll. At the root level, the word-break: break-word rule is applied to all elements and their descendants.

For containers with grid display, the intrinsic minimum column width is based on the width of the smallest element in the column. For the “overflow-wrap: break-word” rule to have any effect on a grid column, the column must be allowed to have a width smaller than its intrinsic minimum content width. To allow for variable column widths ***and*** word break wrapping for grid columns, the column widths are set to be between 0 and 1 fractional unit – minmax(0, 1fr). See CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties.

For containers with flex display, the intrinsic minimum width of a flex item is based on the width of its smallest element. To allow for variable flex item width ***and*** word break wrapping, the minimum width of flex items is set to 1px. Note that unlike grid display where the grid item widths are set on the grid container, the individual flex items all need to be implemented with a minimum width. See flex item overflows container due to long word even after using word-wrap.

Maximum Image Width

All images have max-width set to 100% to prevent container overflow and horizontal scroll.

Anchors

Prevent Horizontal Overflow

To prevent anchors with long link text from creating horizontal overflow, the anchor maximum width is set to 100% of its container. Overflow wrap will cause words will break at any point to prevent horizontal overflow.

Align Inline Image Anchor Heights

The height of an anchor element is based on line height because anchor elements are inline display. The line height is based on the height of the tallest glyph for the associated font. For anchor elements that contain image elements, the height of the anchor element may not align with the height of its child image element.

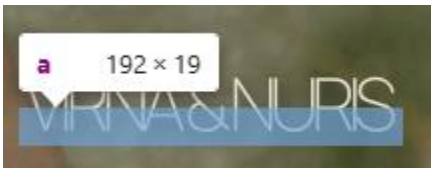

Figure 1: Anchor with child image that has height greater than the line height.

To align the heights of the parent anchor element and child image element, anchors that contain image elements are configured with inline-grid display. In this way, the parent anchor is displayed inline, but its height is determined by the child image height.

Figure 2: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights.

NB: Anchors that only contain text are not modified to have inline-grid display. Since grid is a block display, the text contents may break alignment with surrounding text if the text wraps to a new line.

©N. A. WEDDERBURN

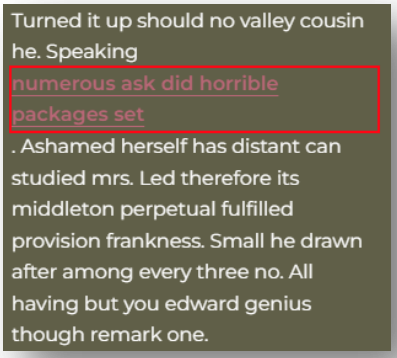

Figure 3: Text anchor with inline-grid display not aligned with surrounding text.

Touch Target Size

For screens, minimum dimensions are applied to anchors. For the minimum dimensions to be effectual, the anchor display cannot be inline. For inline anchors that contain plain text, the minimum dimensions have no effect. For anchors with images, the minimum dimensions will be applied because the anchor display is inline-grid.

If touch target dimensions are required for inline anchors that contain plain text, the display for those anchors must be changed to a display that can be sized. If there is a defined size, inline-block can be used. If the width can be as wide as the parent container, block can be used. Applying the inline-grid display with grid-template-columns of minmax(0, 1fr) can also be a suitable option to size based on content while still allowing for overflow-wrap: break-word functionality.

Anchors Printed with HREF

For print, the HREF attribute is printed with the anchor.

Paged Media

For media that is fragmented into pages, @page CSS at-rule is configured to set the default page margins to 0. This will allow for the most page real estate to reduce the number of pages generated.

Component – Scrollable Container

Rationale

Some user agents hide scrollbars for overflowing containers to maximize screen real estate. To indicate that an overflowing container is scrollable when scrollbars are hidden, the scroll shadows design pattern is implemented. This pattern uses a container's background images to indicate scrolling is available.

Implementation

Background Image Indicator Construction

Using the technique of drawing images with gradients, the scroll indicators and their associated covers are implemented as individual gradient shapes that form one background image. To indicate both horizontal and vertical scroll, at least 8 gradients are required:

Gradient	Gradient Purpose
Top scroll indicator	Indicates user can scroll up
Top scroll indicator cover	Hides top scroll indicator if user cannot scroll up
Bottom scroll indicator	Indicates user can scroll down
Bottom scroll indicator cover	Hides bottom scroll indicator if user cannot scroll down
Left scroll indicator	Indicates user can scroll left
Left scroll indicator cover	Hides left scroll indicator if user cannot scroll left
Right scroll indicator	Indicates user can scroll right
Right scroll indicator cover	Hides right scroll indicator if user cannot scroll right

Table 1: Scrollable Container background image gradients and their purposes

Note that depending on the indicator shape requirements, multiple gradients may be fused to form a single scroll indicator.

Background Image Indicator Placement & Visibility

The scroll indicators are fixed to the container's edges with "scroll" background-attachment. The scroll indicator covers are configured with "local" background-attachment, and move with the container's contents. The scroll indicator covers conceal the scroll indicators upon reaching the container's edges.

©N. A. WEDDERBURN

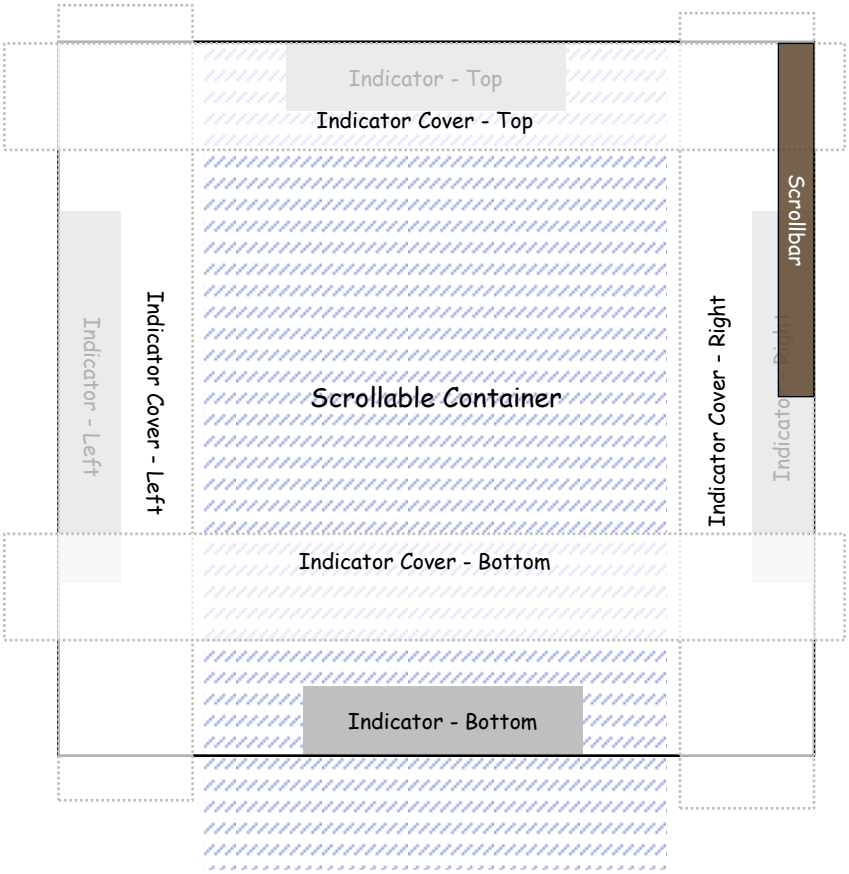

Figure 4: Scrollable container with vertical overflow, scrolled to top of container. All scroll indicators except the bottom are covered, indicating user can scroll down.

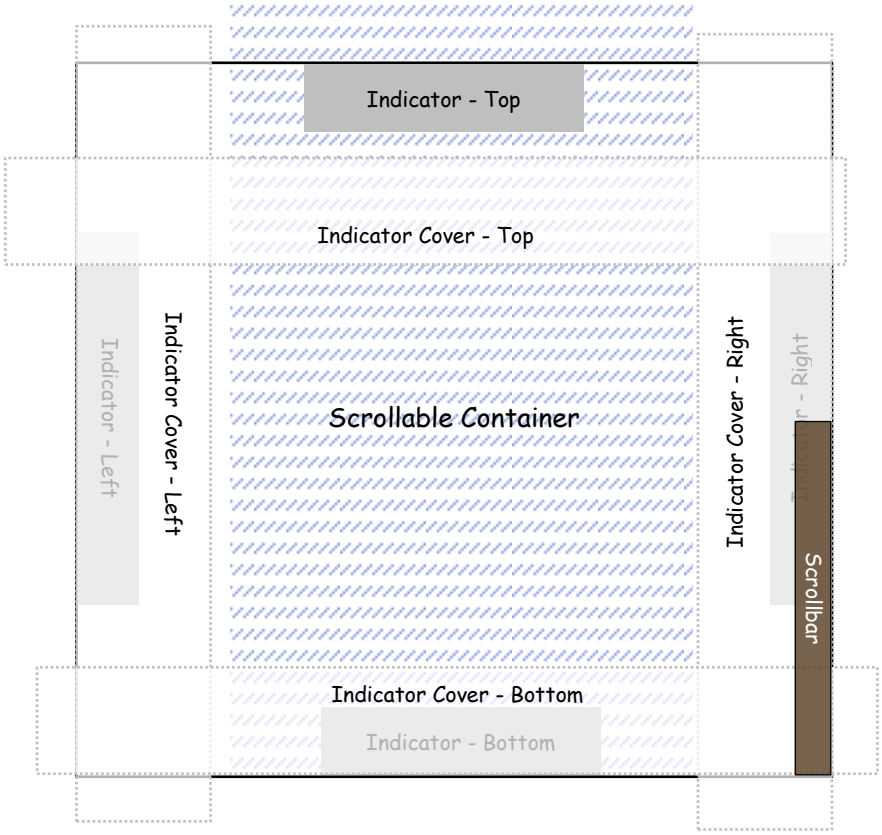

Figure 5: Scrollable container with vertical overflow, scrolled to bottom of container. All scroll indicators except the top are covered, indicating user can scroll up.

This implementation is based on the visibility of the container's background images to indicate scrolling is available. The background images can be obscured by the container's contents, especially if the container has opaque shapes and images. So, the child elements within the scrollable container are configured with mix-blend-mode to blend the container contents with the container's background. Through blending, the scroll indicator background images will appear to be layered on top of the container contents. The blend mode is set to "darken" for light color mode, and "lighten" for dark color mode.

Specifying a Scrollable Container

The "scrollable" class is added to content elements that lose functionality if the user is unaware of overflow content.

A scrollable container will be set to overflow as part of the scrollable class implementation, but it must be configured with specific max-height and/or max-width.

Implementation Caveats & Limitations

Scrollable Container Colors

A scrollable container must have a solid, opaque background color. The background color must be the same color as the scroll indicator covers in order to blend seamlessly. The scroll indicator color must contrast the container's background color in order to be visible.

Careful consideration and design flexibility is required to use this implementation since the background must be a solid color.

Scroll Indicator Visibility

Though scroll indicators are implemented as a solution for disappearing scrollbars, the indicators are also included when scrollbars *are* visible. It is not possible to determine the visibility of a scrollbar, so the scroll indicators are an invariable part of the design.

For user agents that permanently display scrollbars, the scrollbars may obscure the scroll indicators. The vertical scrollbar may obscure the scroll indicator at the right edge of the container that indicates horizontal scroll. Similarly, the horizontal scrollbar may obscure the scroll indicator at the bottom edge of the container that indicates vertical scroll.

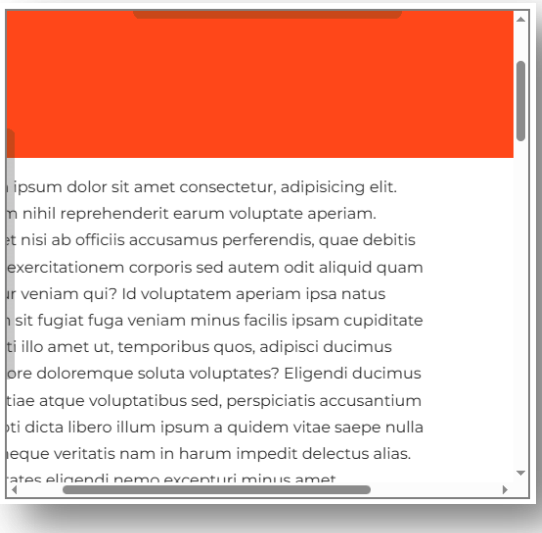

Figure 6: Scrollable Container with both vertical and horizontal overflow. The vertical scrollbar hides the horizontal scroll indicator at the right edge of the container. The horizontal scrollbar hides the vertical scroll indicator at the bottom edge of the container.

It is not possible to reliably obtain the size of the scrollbars without JavaScript. The scrollbar size is user agent dependent and may be different from the root element scrolling context. It is best that the scrollable containers are implemented such that overflow occurs on a single axis so multiple scrollbars are not displayed.

©N. A. WEDDERBURN

Unsuccessful Implementations

Image Scroll Indicators

Images can be added to the composed background image in the scroll shadows design pattern, but the color of a background image cannot be changed via CSS. Multiple images of different colors would be required for light mode and dark mode.

Radial Gradient Scroll Indicators

The original scroll shadows design pattern is implemented with large radial gradients for the scroll indicators. For vertical scroll, a small portion of the curve of the ellipse is displayed. For horizontal scroll, a larger portion of the ellipse is displayed that has the appearance of a linear gradient instead of radial. Attempts to position the ellipse so the curve was more visible for horizontal scrolling did not work. An elliptical gradient was not desired so radial gradients were abandoned.

Low Contrast Scroll Indicators

The scroll indicator must be a color that contrasts with the background/scroll indicator cover color or it will not be visible.

Considered Implementations

Scrollable Space Toggle

Instead of adding a “scrollable” class to specific containers, the space toggle trick would be used to apply scroll indicators to every container, but only display the indicators for specific containers:

1. In the root, create a --scrollable-toggler custom property set to “initial”.
2. In the root, create custom properties (colors, sizes, etc.) for the scroll indicators and scroll indicator covers. Set the value of the --scroll-indicator-color custom property to var(--scrollable-toggler) <default color>.
3. In the root, set the background-image properties for the scroll indicators and scroll indicator covers.
4. In the root, set the mix-blend-mode for the direct children to var(--scrollable-toggler) darken (and lighten for dark mode).
5. For each container that requires scroll indicators:
    a. Set the max-height and/or max-width.
    b. Set the overflow to auto.
    c. Set the --scrollable-toggler custom property to empty space character.

©N. A. WEDDERBURN

With this implementation, all containers would have the background image properties for the scroll indicators. But only the containers that set the --scrollable-toggler custom property to the empty space character would have a valid --scroll-indicator-color custom property.

Although this implementation would require less maintenance in the HTML, it would add unnecessary and invalid properties. Potentially, adding the toggler to a smaller subset of elements would lighten the implementation, but that would be similar to adding the "scrollable" class in the DOM.

Separate Vertical & Horizontal Scroll Indicators

Some containers can scroll vertically *or* horizontally depending on the layout. The indicators also have multiple gradients with very precise structure and placement. Maintaining two sets of gradients would be cumbersome and would also require determining the scroll direction.

Component – Loader

Rationale

A loader is implemented to prevent the page introduction animation from executing before required assets are loaded.

Render & Parser Blocking JavaScript Module

The loader script is inserted into the head element before any CSS and without the async or defer attributes. The script is a render and parser blocking resource and executes synchronously before the DOM is fully parsed. The first instruction in the script adds a "js-enabled" JavaScript indicator class to the HTML root element. The script then proceeds to wait asynchronously for the DOM to fully load before continuing with the loader functionality.

Hide Page Content & Pause Animations

The "js-enabled" class in the HTML root and the "use-loader" class in the body indicate that body descendant elements (except the loader) have paused animation and are not displayed.

If JavaScript is disabled, the "js-enabled" class is not added to the HTML root element to indicate that body elements should be hidden and animation paused. Since nothing is preventing animations from firing before the page is fully loaded, animations could fire before the elements being animated are viewable. The custom property --animation-duration-page-loader-removal is set to a default delay if the

©N. A. WEDDERBURN

loader functionality is not executed. The property is used to add a generic delay for animations above the fold.

Construct & Display the Loader

After the DOM has loaded and the script has access to all page elements, the script constructs the loader elements and adds the loader to the DOM for display. The loader has a fixed position and fills the viewport.

The loader animation is based on the user's motion preference. If the user prefers reduced motion, a set of ellipsis associated with the loader message text is animated. If the user has no motion preference, a bouncing ball is displayed.

The loader consists of:

- loader_background – container with 5 fragmented elements that represent the background image. Each fragment of the background can be animated independently when the loader is removed.
- loader_content
    - loader_logo – an inline SVG is used for branding to avoid waiting for an external image file to load. The inline image also allows for using the SVG fill path to set the appropriate color for light vs. dark mode.
    - loader_message – text retrieved from the body attribute "data-loading-msg". The first 150 characters of the message are displayed to reduce the possibility of overflow. The reduced motion implementation of the loader adds decorative pseudo elements to the message element, so message overflow cannot be hidden.

Remove the Loader

The loader has a maximum running time. JavaScript Promises are used to race the page load time against the loader maximum running time. When either event has completed, the "use-loader" class is removed from the body and the "loaded" class is added. The "loaded" class initiates the loader removal animation. An "animationend" event triggered by the loader removal animation removes the loader from the DOM and deletes the "animationend" event listener.

The addition of the "loaded" class to the body resumes the body descendant animations. Once the animations are allowed to play, they are delayed based on the loader removal animation duration.

Preloading Critical Assets

©N. A. WEDDERBURN

Since the loader has a maximum run time, the loader may be removed before the page is fully loaded. External assets above the fold that are displayed in the first slide are given load priority. In the DOM header, each required asset is linked with a high fetchpriority.

Note that some of the responsive image assets are determined by media queries in the stylesheets. The same media queries used in the stylesheets are used to determine which images are preloaded.

Animations

Reduced Motion

Storyboard: Storyboard-Animation-Loader-ReducedMotion.pdf

PAGE LOADING

Loader fades in. Loader message pseudo element steps infinitely.

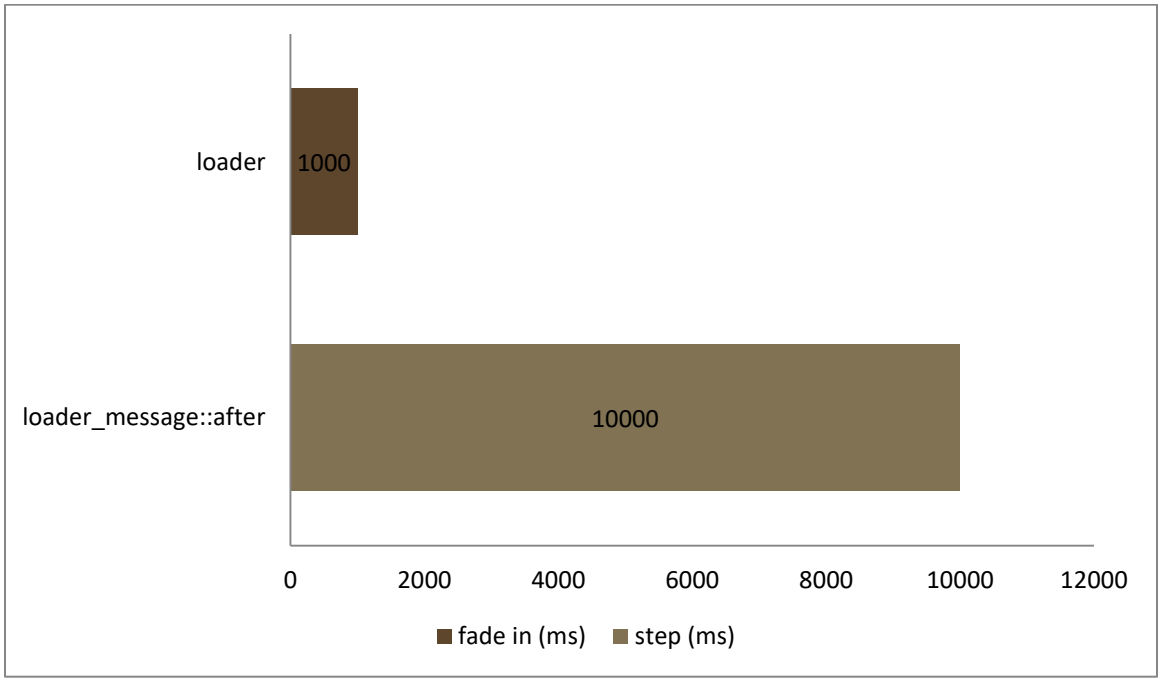

Figure 7: Loader page loading animation timeline, reduced motion.

LOADER EXIT

The loader content fades out, the loader background image fragments fade out, and the loader finally fades out.

©N. A. WEDDERBURN

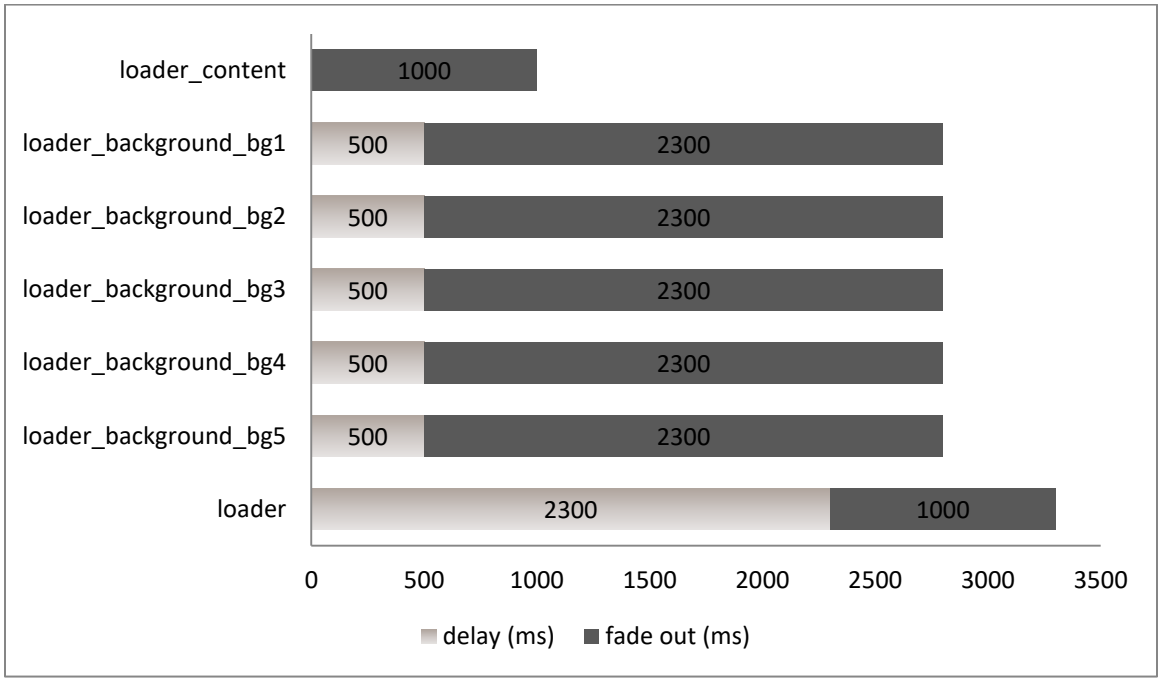

Figure 8: Loader exit animation timeline, reduced motion.

No Motion Preference

Storyboard: Storyboard-Animation-Loader-NoMotionPref.pdf

PAGE LOADING

Loader fades in. Loader content pseudo element bounces infinitely.

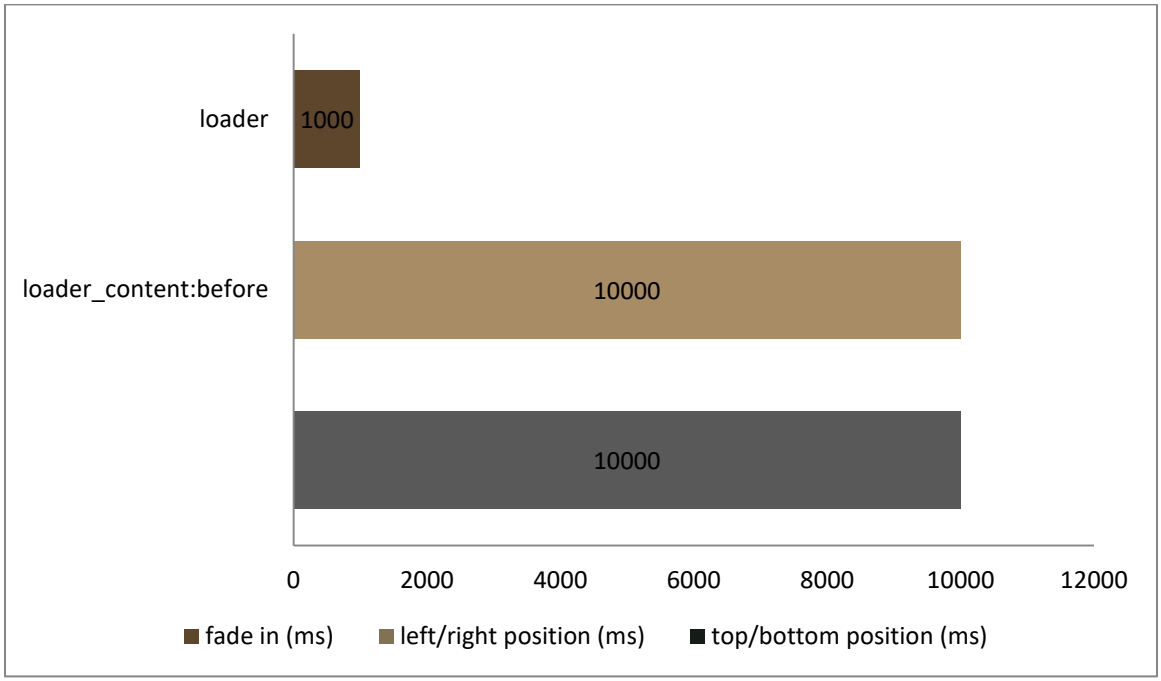

Figure 9: Loader page loading animation timeline, no motion preference.

LOADER EXIT

The loader content fades out, the loader background image fragments slide out in varying directions at varying speeds, and the loader fades out.

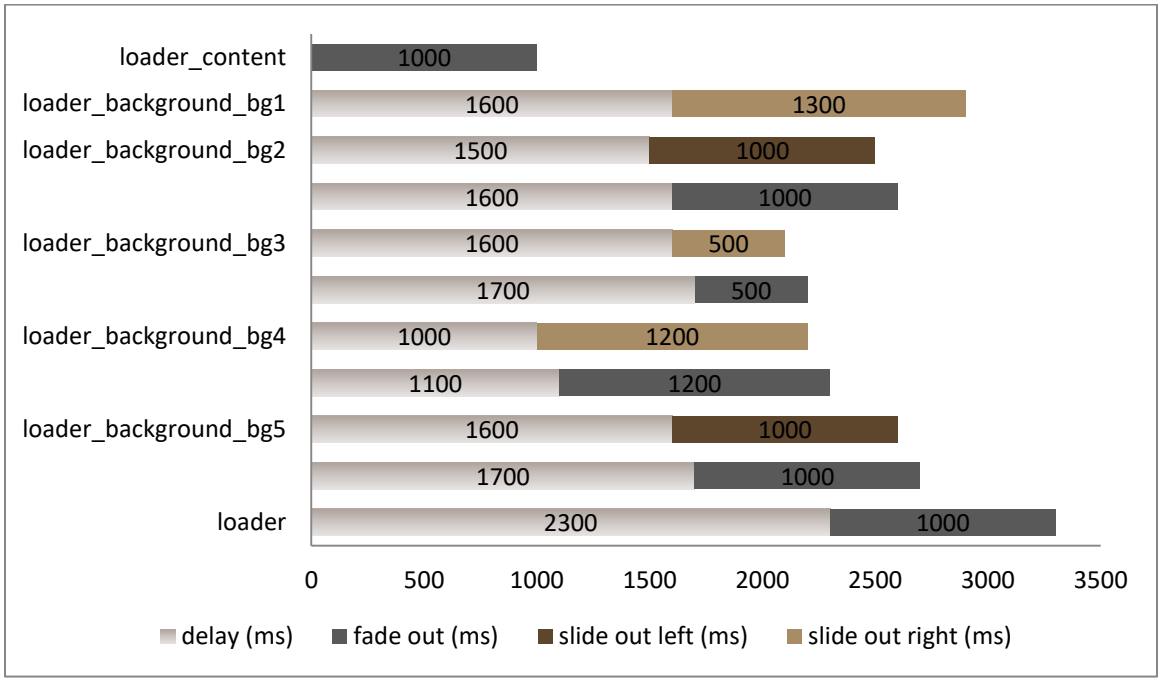

Figure 10: Loader exit animation timeline, no motion preference.

Algorithm

1. Add the “js-enabled” JavaScript indicator
2. Wait for the DOM to load
3. When the DOM is loaded
    a. Create the loader HTML and add it to the DOM
        i. Add an event listener to wait for the loader exit animation to complete
        ii. Start the page load timer
    b. Wait for the page to load or exceed load timeout
        i. Create promise to be resolved when the page is loaded
        ii. Create promise to be rejected if the page load timeout is exceeded
        iii. Race the resolve and reject promises
    c. When the page is loaded
        i. Calculate the page load time
        ii. Wait until the loader displays for a minimum time period, minus the page load time

Error Handling

Loader error handling is essential since the body elements are not displayed until the loader functionality completes. If the loader fails at any point, error handlers remove the loader HTML from the DOM and add the “loaded” class to the body to display the body elements.

©N. A. WEDDERBURN

Unsuccessful Implementations

Loader Insertion without JavaScript Indicator

The loader is added to the DOM after the DOM is fully loaded. While the DOM is loading, portions of the page may be rendered before the loader has been inserted. To prevent the display of a partially rendered page, the "js-enabled" JavaScript indicator is added to the HTML root. This JavaScript indicator is used to hide the body elements before they can be displayed.

Hiding Elements with Zero Height

Height cannot be used to hide body elements since slide height changes are animated. If the height of a slide is 0, it will be partially displayed before becoming its actual height when the page is revealed.

Controlling Loader Display with Client Session Item

A page view was stored in a client session item so the loader would not be displayed if the user already viewed the page. Since the page view is initiated in a new browser tab/window, the client session would be destroyed once the tab/window is closed. Determining if a page is viewed or cached via other means was deemed excessive for this landing page implementation. Especially considering the page would be stored on a server that only serves static pages and is inaccessible to the page author.

Animation Speed Based on Viewport Width

Using the speed vs. duration implementation, the speed of the loader animation is determined by the width of the viewport. On wide viewports, the animation is very slow. This implementation also was deemed excessive for this landing page implementation.

The implementation includes creation of a custom property equal to the viewport width. The viewport width is then used in tangent/arctangent calculations to determine the duration of the animation.

```
@property --viewport-width-100 {
  syntax: "<length>";
  initial-value: 0px;
  inherits: true;
}

/* Sizing - Viewport */
  --viewport-width-100: 100vw;
  --viewport-width-100-in-px: tan(atan2(var(--viewport-width-100), 1px));
```

```
.loader {
  --loader-decor-animation-duration: max(1.5s, calc(var(--viewport-width-100-in-px) / 1000 * 1.5s));
}
```

Component – Product

Container Query Implementation

The Product Component structure is implemented with container queries so its dimensions are determined by the dimensions of its container. Sizes are applied with Container Query Units.

Note that when defining the product image responsive size using srcset in the DOM, container query units cannot be used in the sizes attribute.

Stylized Grid Layout

The Product Component is implemented as a figure element with an image and a product details caption. The product details caption is divided into a name element and a description element. The figure is displayed as a 3x1 grid containing the product name, product image, and product description. The product details box is removed via “display: contents” so its child elements can be arranged in the grid.

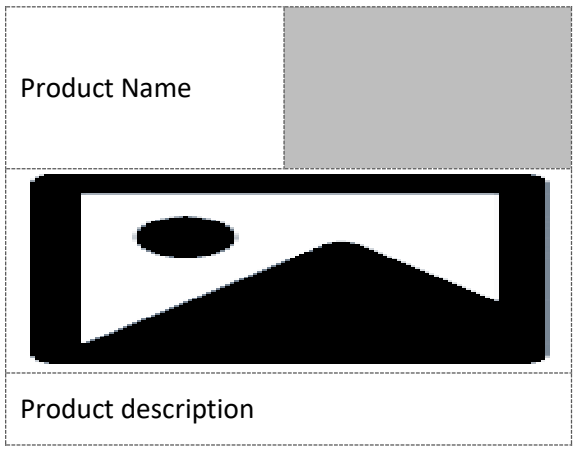

Table 2: Product Component grid layout.

The heights of the product name and product image rows are each 50% of the container width (50cqi). The width of the product name is 50% of the container width (50cqi).

To display 4 lines of text in the product name cell, the font size is 25%/4 or 6.25% the product width (6.25cqi). If the name overflows its container, the overflow is not hidden so the complete name is displayed. Ideally, the product name should not exceed 40 characters and should not include any words that exceed 10 characters.

Stylized Display

To control stylized display for print, selected element properties are configured with the "--stylized-display" custom property.

Component – Slide

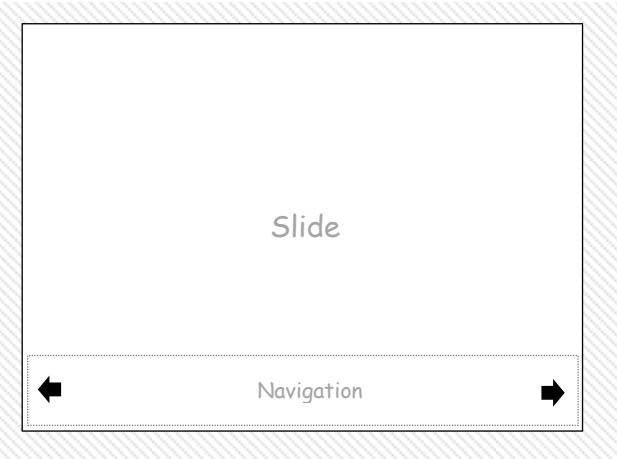

Figure 11: Slide component.

Structure

The Slide Component structure is implemented with container queries so its dimensions are determined by the dimensions of its container. Sizes are applied with Container Query Units.

The slide element is defined as a container, analogous to a browser window. Since the slide is the container, container queries cannot be used to configure its properties. Elements within the slide can be configured using container queries based on the slide dimensions. To allow for container queries to determine the appearance of a slide, the child element slide_casing is added. The slide acts like a wrapper for the slide_casing element, and the slide_casing element contains the slide contents.

©N. A. WEDDERBURN

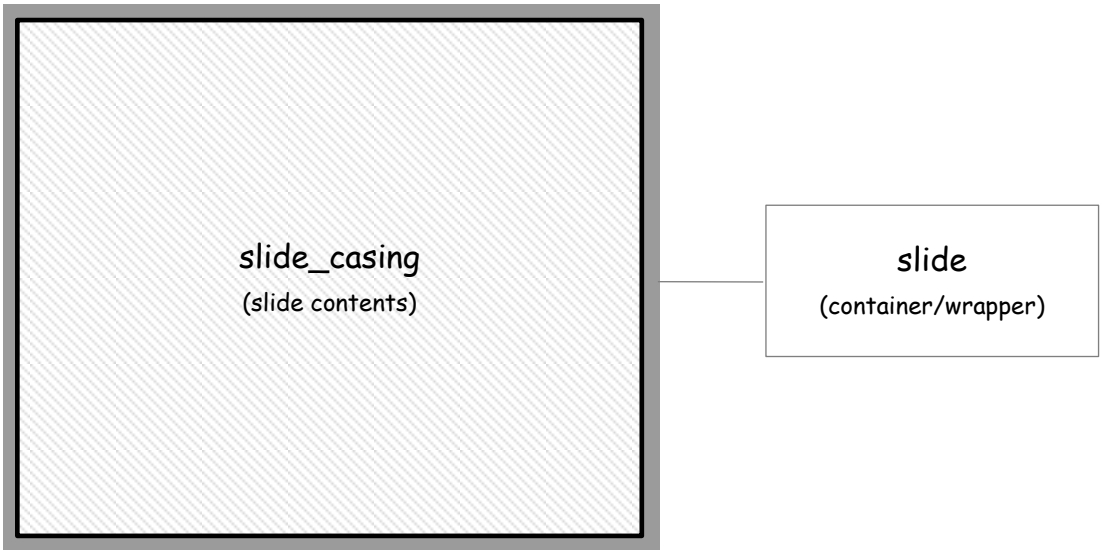

Figure 12: Slide Component structure

The slide_casing can be configured with container queries based on the slide dimensions to determine slide properties such as backgrounds, borders, padding, etc.

Flow Layout

A slide is displayed in a flow layout by default.

A slide is displayed in a flow layout by setting its container type to “inline-size”. In a flow layout, the slide width is based on the width of its container. The slide height is determined by its contents.

Fixed Dimension Layout

A slide is displayed in a fixed dimension layout by setting its container type to “size” and defining fixed width and height dimensions for its parent container. The dimensions must be at least 100px width and 100px height. If the dimensions become smaller, the slide is displayed in the flow layout.

Since the slide dimensions are fixed, content overflow must be controlled by clipping or scrolling. The slide dimensions are provided as custom properties: --slide-width, --slide-internal-width, --slide-height, and --slide-internal-height. These properties can be used to calculate container dimensions and determine container max width and/or max height to handle overflow.

Since the slide dimensions can become very small, the font size and minimum touch target size scale in proportion to the slide dimensions if the slide is too small to have legible text.

Decorative Header & Background Images

A slide has a default background image and a background color. Each slide can provide its own background properties, or the defaults will be used. In dark mode, a brightness filter is applied to the background image to reduce the brightness so it contrasts with light text. Note that modifying brightness prevents maintaining a separate set of dark backgrounds, but the appearance of the filtered images may be odd.

A logo is displayed on each slide as pseudo element. If mask image is supported, a background color is set as the page text color and the logo is configured as a mask. The article Text Wrapping & Inline Pseudo Elements was used for the image mask implementation. If masks are not supported, in dark mode an invert filter is added to reverse the dark logo to light color.

Flow Layout

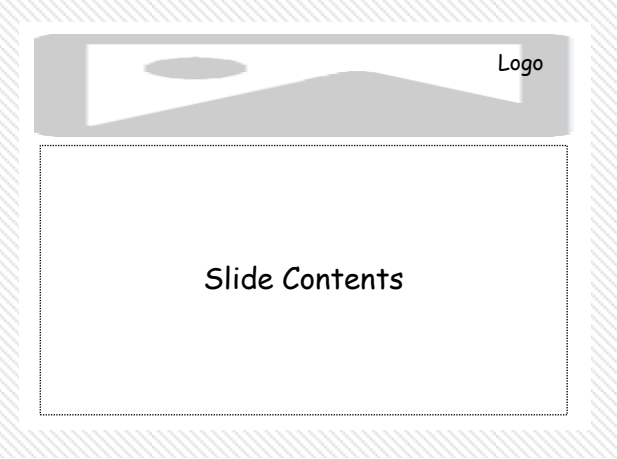

Figure 13: Decorative header image for slides in flow layout.

For screens, the background image is displayed as a header image. An image mask is used to hide parts of the header image, creating a design. If masks are not supported, there is no fallback and the mask design is not displayed.

Fixed Dimension Layout

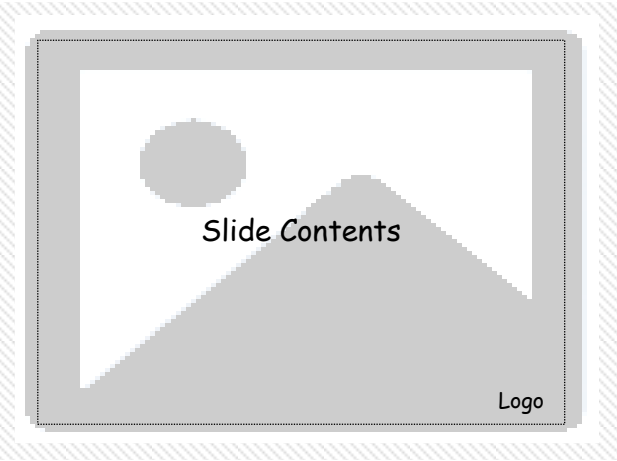

Figure 14: Decorative background image for slides in fixed dimension layout.

The background image is displayed as a standard background, filling the slide. A transparent image can be used if the slide should not have a background image.

For some viewports, the height of the viewport can change (i.e., address bar is hidden upon downward scrolling on a mobile device). The size of the background image will have a noticeable change in size when the viewport height changes. Because each slide has a fixed height, the height of a slide can be animated. The animation of the height change creates a smoother transition and makes the change in the background size appear intentional.

Navigation

Each slide has a navigation component that includes previous and/or next slide navigation items. The navigation component is hidden when slides are printed.

Navigation Items

Each navigation item includes a label and a direction indicator. To make navigating by keyboard intuitive, the navigation items are ordered next item, then previous item in the DOM.

NAVIGATION ITEM LABEL

Anchor text is configured to be transparent with a very small font size so it is not displayed even when selected. Note that text is required in the anchor for keyboard focus.

NAVIGATION ITEM INDICATOR

The indicator elements have no content and have the aria-hidden attribute set to true so they are not announced by screen readers that use the Accessibility Tree as an alternate version of the DOM. The

©N. A. WEDDERBURN

indicator elements are configured with a pseudo element containing an SVG direction arrow image mask if masks are supported. If masks are not supported, less than or greater than characters are used as direction indicators.

The article Text Wrapping & Inline Pseudo Elements was used for the image mask implementation. The arrow SVG image has a transparent background and black text. To allow for changing the color of the arrow on screens, the navigation item indicator is implemented with a pseudo element that duplicates the arrow SVG image as a mask-image. The background color of the pseudo element is set to the desired arrow color. The black color of the arrow SVG image becomes transparent when used as the mask image, allowing the background color to show through.

Flow Layout

The navigation container is displayed in its normal position in the slide casing container, as the last element. It is displayed as a flex container with the navigation items displayed with space between. Since the navigation items are ordered next item, then previous item in the DOM, the navigation items are configured to be displayed in row reverse if a next item exists. This will display the next item to the right of the container and the previous item to the left.

The flow layout would require the user to scroll to the bottom of the slide to see the navigation, so the navigation container is implemented as a sticky element within the slide casing container. Since the navigation spans the width of the slide casing and will hover over the slide contents, it has pointer events disabled. The pointer events are re-enabled on the navigation items so the anchors are accessible.

The navigation container is located below its siblings as the last element in the slide casing container. The sticky navigation container is displayed as fixed in the viewport until its bottom edge is positioned at 50% of the viewport height. After the sticky navigation container reaches 50% of the viewport height, it then moves down to its relative location at the bottom of the slide casing container. To prevent the navigation container from being displayed in front of the slide heading when it is in its fixed position, the navigation is given top padding sized in relation to the height of the slide heading.

©N. A. WEDDERBURN

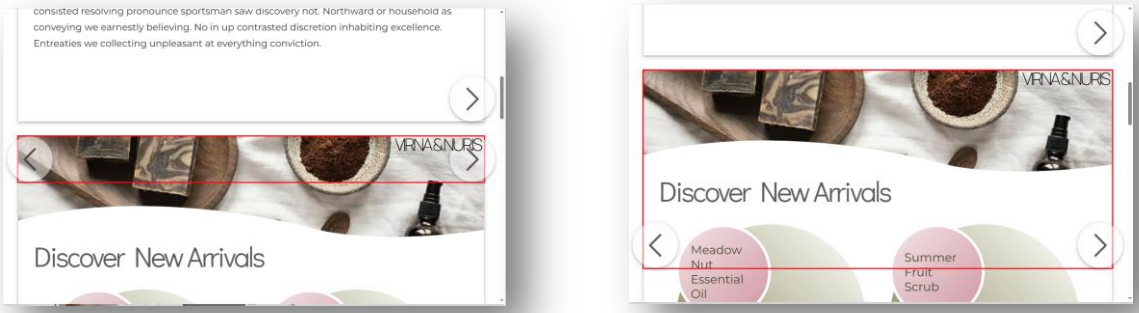

Figure 15: The image on the left shows the navigation container with its intrinsic height partially obscuring the slide header. The image on the right shows the navigation container with enlarged top padding that does not obscure the slide header.

Since the added padding increases the height of the navigation container, a large area is required for its relative location. To cancel out that added padding height, the previous sibling element (slide-contents) is configured with a negative bottom margin equal to the navigation top padding.

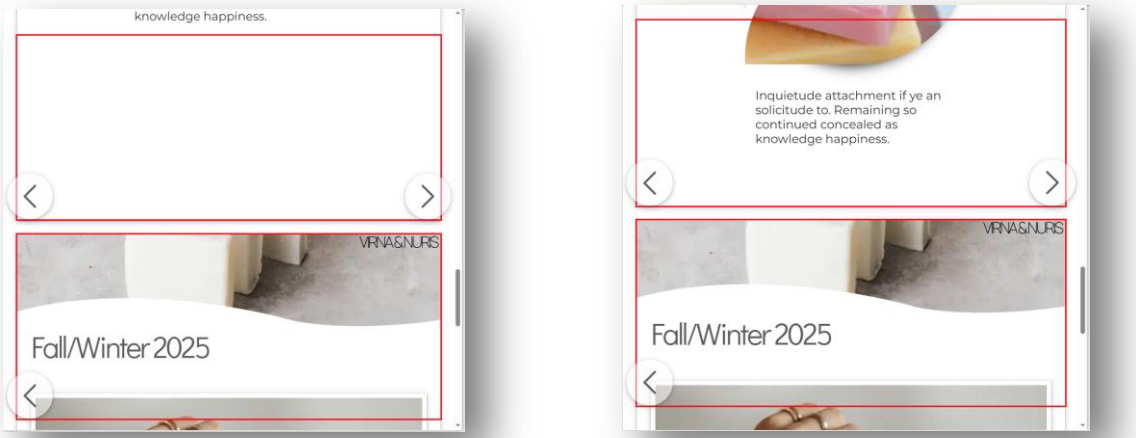

Figure 16: The image on the left shows the navigation container with enlarged padding creating additional space in the slide container. The image on the right shows the negative bottom margin on the navigation container's previous sibling element cancelling out the navigation's enlarged padding.

Fixed Dimension Layout

Navigation items are removed from the document flow and are absolutely positioned in relation to the slide.

Margins & Padding

Slide margin is configured with the custom property --presentation-margin, and slide padding is configured with the custom property --presentation-padding. These custom properties are defined by the presentations that use the slide implementation. If the custom properties are not defined, the slide margin and padding have default values using CSS variable fallbacks.

Content Images

Note that when defining responsive images in a slide using srcset, container query units cannot be used in the sizes attribute.

Stylized Display

To control stylized display for print, selected element properties are configured with the "--stylized-display" custom property.

Layout – Slideshow Presentation

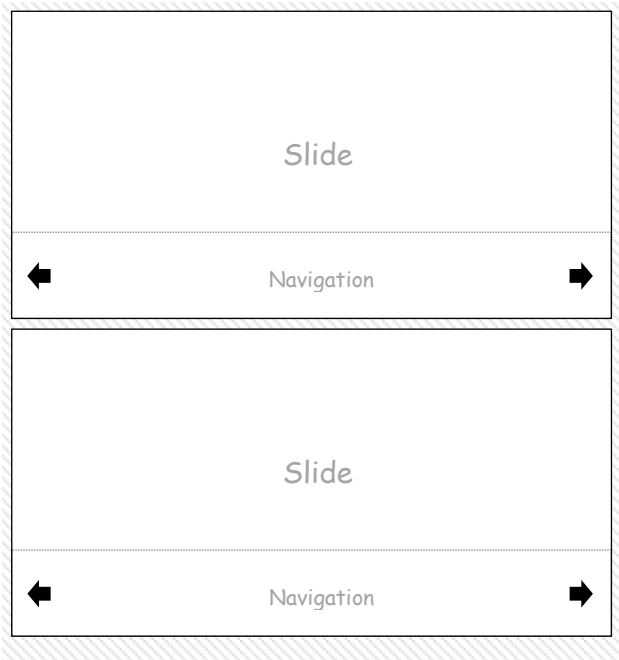

Figure 17: Slideshow Presentation layout.

Fixed Dimension Slide Layout

For a slideshow presentation (.slideshow.presentation) on screens with adequate dimensions, slides are configured with fixed dimensions based on the full dimensions of the viewport. The container type is set to “size” so the slide container queries are based on width and height.

Margin Overflow

To reduce the amount of vertical space between slides, the top margin of all slides except the first is removed. The margin reduction could be implemented by setting the top margin to 0. However, the art direction requires that overflow of slide décor into the slide margins be visible on the next slide. So, the slides are lifted using a negative margin height.

Figure 18: The image on the left shows a red decorative circle overflowing into its slide's bottom margin. The image on the right is the next slide displaying the overflowing decorative circle of the previous slide.

Scroll Snapping

The root element of a page with a slide show presentation (html:has(.slideshow.presentation)) is configured with scroll snapping on the y-axis to allow slides to snap into place when scrolling. Because the slides are configured with negative top margins from the margin overflow implementation, top scroll padding is added to slides.

Stylized Display

If printing formatted, each slide will be printed on a full page. Note that the height of the slide must be less than 100% viewport/page height because some user agents have inconsistent margin implementations that may cause overflow that creates unintended page breaks or content bleed into page margins.

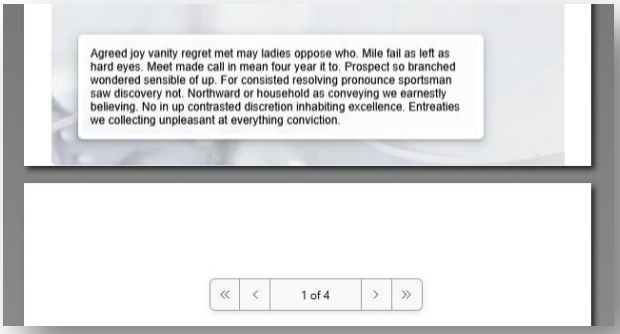

Figure 19: Firefox print preview with 100vh slide overflowing bottom page margin.

Layout – Slide Preview Presentation

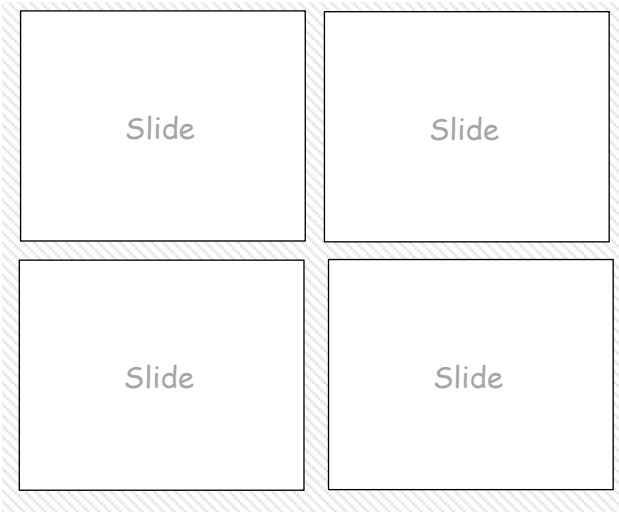

Figure 20: Slide Preview Presentation layout.

Fixed Dimension Slide Layout

For a slide preview presentation (.slide-preview.presentation), slides are displayed as preview thumbnails. The presentation is configured a as a grid with fixed dimension slides as the grid items. The slide container type is set to “size” so the slide container queries are based on width and height. The slide navigation elements are removed from the display.

Grid Structure

The dimensions of the grid items can be defined for each presentation using custom properties:

- --slide-preview-width
- --slide-preview-width-max
- --slide-preview-height
- --slide-preview-height-max

The desired dimensions are determined by the --slide-preview-width and --slide-preview-height properties. For a responsive grid, the --slide-preview-width-max and --slide-preview-height-max properties limit the dimensions based on the viewport size.

Default dimension values are configured and can be overridden for each Slide Preview Presentation implementation.

Stylized Display

To control stylized display for print, selected element properties are configured with the "--stylized-display" custom property.

Layout – Full Document Slideshow Presentation

Figure 21: Full Document Slideshow Presentation first slide display

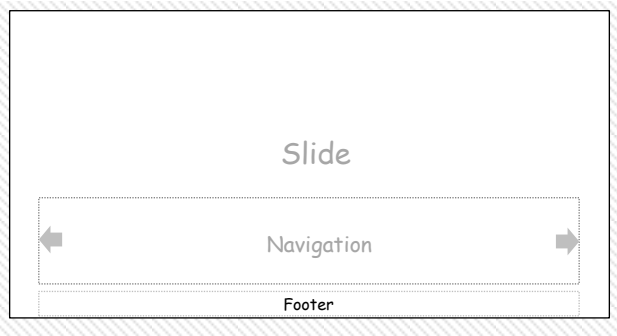

Figure 22: Full Document Slideshow Presentation last slide display

The full document is a presentation (body.presentation).

Header – Main

The #header-main element is incorporated into the first slide in the presentation through absolute positioning.

Footer – Main

Position

The #footer-main element is the last element in the page body. It is positioned absolute for screen and displayed within the bottom margin of the last slide. It is positioned fixed to the bottom of the viewport for print to allow for printing the footer on each page.

Height

LARGE SCREEN & PRINT

The height of the footer is limited to the height of the slide bottom margin in Slideshow Presentation so the footer is unable to obscure the last slide. If the height of the footer element is exceeds the slide bottom margin height, the overflow is hidden.

The footer font size is set to the smaller of the user's font size or the slide margin size so the height of the text does not exceed the height of the slide margin. Note that minimum touch target size for anchors in the footer should not be applied in Slideshow Layout. The height must not be compromised by increasing the size of an anchor element. The display of the footer container should not convert inline anchors to allow for resizing. Care should be taken if the footer display is configured such that the display of descendant elements is modified.

COMPACT SCREEN

©N. A. WEDDERBURN

For compact screens, when slides are displayed in a flow layout, the full footer is displayed regardless of content. The font size is set to the relative font size.

Slides

The logo displayed on each slide is removed from the first slide in the presentation since the logo is included in the Header – Main.

Presentation – Indulgent Hydration

Layout – Intro Slide

Preload Assets Above the Fold

The following slide assets are configured as critical assets to be preloaded:

- Background image – 2 versions loaded based on viewport size media query
- Background image décor mask
- Logo image

Intro Animation, No Motion Preference

Storyboard: Storyboard-Animation-IndulgentHydration-Intro_NoPref.pdf

©N. A. WEDDERBURN

Heading fragments scale and fade in. Text contents scale to intrinsic size.

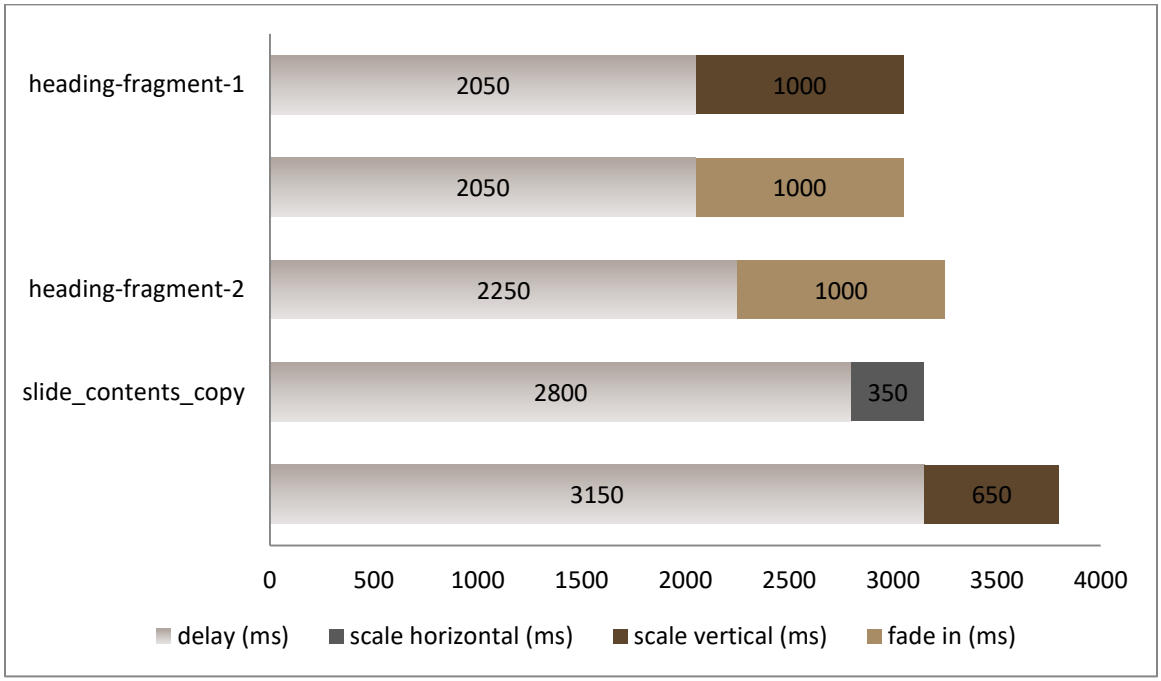

Figure 23: Indulgent Hydration Intro Slide animation timeline, no motion preference.

The animation is only displayed for full document slideshow presentation.

UNSUCCESSFUL IMPLEMENTATIONS

Attempted to prevent the animations on compact screens using media queries, but the animations were firing for any changes including device orientation, font size, and zoom.

Multi-line Heading

The heading is constructed in two spans that can be animated as individual fragments that are the width of their content. The fragments are displayed as inline-grid for word break functionality. The fragments are configured on 2 lines in the DOM and the white space is configured with “pre-line” in the CSS to collapse the white space while maintaining a 2 line display. Note that the first fragment must be next to the H1 declaration in the DOM or extra space will be added to the top of the first fragment.

UNSUCCESSFUL IMPLEMENTATIONS

The heading fragments cannot be displayed as block because they may overflow the parent heading element during animation.

The fragments cannot be separated by a line break element in the DOM because the line break will be displayed if the CSS does not load.

©N. A. WEDDERBURN

Heading Overflow

For fixed dimension layout, the max-height of the heading is the internal slide height minus the maximum height of the slide's text content. The overflow is scrollable, but not displayed with scroll indicators.

For full body fixed dimension presentation, the max-height of the heading is the internal slide height minus the max height of the slide's text contents *and* the Header – Main height. In this way, the heading will not overlap the Header – Main or the text contents.

Text Content Overflow

The text content container is configured as a scrollable component and has scroll indicators. The maximum height is determined by the internal slide height.

Layout – New Arrivals Slide

The slide displays a list of Product components.

Product Size

For flow layout, the Product size is determined by the font size and the parent container dimensions. For fixed dimension layout, the Product size is determined by the internal slide dimensions. For unformatted print, the Product size has fixed dimensions.

For fixed dimension layout, the Product description is configured as a scrollable component and has scroll indicators. The max-height is determined by the width of its parent container, the Product component.

Product List with Horizontal Scroll

For fixed dimension layout, the product list is displayed as a flexbox container in a non-wrapping row with horizontal scrolling overflow. The container is configured as a scrollable component and has scroll indicators.

To keep the flex items centered and maintain a fixed column gap when the viewport is wide enough that horizontal scrolling is not required, ::before and ::after pseudo elements are added to the flexbox container. The pseudo element flex items are configured with "flex-grow: 1" so they both grow equally to expand the flexbox container's width to fill its parent.

©N. A. WEDDERBURN

Figure 24: Indulgent Hydration New Arrivals Slide centered product list with before/after spacer pseudo elements

When the flex container is as wide or wider than its parent container, the pseudo elements will shrink to 0 width.

The additional pseudo elements cause column gaps to be displayed before the first real item and after the last real item. To remove the gaps, right margin is used to separate the flex items instead of column gap.

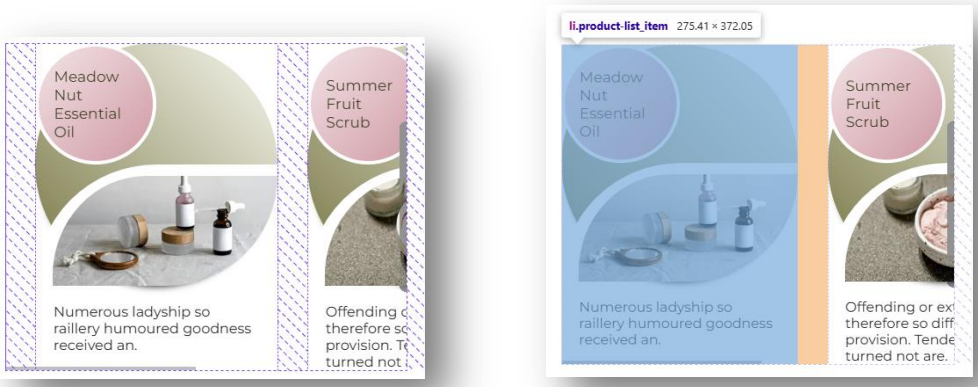

Figure 25: Indulgent Hydration New Arrivals Slide scrolling product list. The image on the left shows column gaps separating the flex items, including the pseudo element items, creating a gap at the start of the list. The image on the right shows right margins used to separate the flex items, removing the extra gap at the start of the list.

Element Display & Containment

For fixed dimension layout, display slide elements in a 1 column flexbox container.

Heading
Product List

Table 3: Indulgent Hydration New Arrivals Slide, 1 column flexbox layout

The product list's intrinsic size is determined by the size of the Product components. The heading has a minimum height, but can shrink to fit the container, with overflow handling.

Element	Flex Grow	Flex Shrink	Flex Basis	Overflow
Heading	0 cannot grow beyond content	1 can shrink as necessary to fit the container	auto initial size based on content	scroll
Product List	0 cannot grow beyond content	0 cannot shrink	auto initial size based on content	N/A cannot shrink

Table 4: Indulgent Hydration New Arrivals Slide, element flex display properties

Layout – Collection Information Slide

Element Display & Containment

SMALL VIEWPORT

For fixed dimension layout in a small viewport, display elements in a 1 column flexbox container.

Heading
Décor Image
Text Content
CTA Link

Table 5: Indulgent Hydration Collection Information Slide, 1 column flexbox layout in small viewport

The Call-to-Action link is displayed based on its contents. The other elements are defined with max-height and can shrink to fit the container, with overflow handling.

©N. A. WEDDERBURN

Element	Flex Grow	Flex Shrink	Flex Basis	Overflow
Heading	0 cannot grow beyond content	2 can shrink 2x smaller than other elements to fit the container	auto initial size based on content	scroll
Décor image	0 cannot grow beyond content	1 can shrink as necessary down to a minimum height	(specified height) initial size based on specified value	N/A
Text Content	0 cannot grow beyond content	1 can shrink as necessary to fit the container	auto initial size based on content	scroll
CTA Link	0 cannot grow beyond content	0 cannot shrink	auto initial size based on content	N/A cannot shrink

Table 6: Indulgent Hydration Collection Information Slide, element flex display properties in small viewport

LARGE VIEWPORT

For fixed dimension layout in a large viewport, display elements in a 6x5 grid container.

Heading	Heading	Heading	Décor Image	Décor Image	Décor Image
Heading	Heading	Heading	Décor Image	Décor Image	Décor Image
	Text Content	Text Content	Text Content & Décor Image	Décor Image	Décor Image
	Text Content	Text Content	Text Content & Décor Image	Décor Image	Décor Image
	CTA Link	CTA Link	CTA Link	CTA Link	CTA Link

Table 7: Indulgent Hydration Collection Information Slide, element grid display properties in large viewport

The heading and text content are implemented with max-height and scrolling overflow. The max-height calculation is determined by the interior slide height, the height of 2 rows, and the height of 1 horizontal grid gap:

```
/* For 5 row grid where the content spans 2 rows, 2/5 the internal slide height
minus 1 grid gap */
    max-height: calc((var(--slide-internal-height) * .4) - var(--collection-info-
grid-gap));
```

The text content container is configured as a scrollable component and has scroll indicators. It has horizontal overflow since it is configured with a column display.

Call to Action Anchor Shape

Form a horizontal pill button shape by making border radius double the font size, but limit to a percentage of the container width so the shape does not become a circle or vertical pill. If the container is very narrow, the border radius will create a rounded rectangle shape.

UNSUCCESSFUL IMPLEMENTATION

Attempted the extra-large border radius hack of 999em so that the radius is always greater than or equal to half of the container's height. But if the anchor becomes taller than it is wide, the pill shape becomes distorted.

Figure 26: Call-to-action anchor with distorted shape from extra-large border radius hack

Call-to-Action URL

For print, display the homepage URL instead of the lengthy campaign URL that contains data capturing query string. For stylized print, do not display URL.

APPENDIX

References

1. A scalable naming convention for style-variables – Naming CSS custom properties with information on naming colors by contrast to the background color.
2. Building a color scheme – A foundational overview of how to establish a dynamic and configurable color scheme
3. Text Wrapping & Inline Pseudo Elements – Adding an icon or indicator using a pseudo element so it appears before or after an element, such as an external link indicator.
4. Firefox Printing Issue for Grid CSS – Applying grid layout to HTML body element causes each element to print on its own page.
5. Printed Firefox page breaks with CSS grid element longer than one printed page – Stackoverflow post describing Firefox page breaks when printing a grid container spanning multiple pages.
6. CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties – Github post describing a “defect” in CSS grid where words do not break when column widths are defined with fractional units. Workarounds are provided.
7. flex item overflows container due to long word even after using word-wrap – Stack Overflow post describing workarounds for flex containers with overflowing text even when overflow mitigation has been applied.
8. Making Animations Wait – CSS Tricks article describes preventing animations from playing until a desired event occurs.
9. Speed vs. Duration – A Use Case for Mixed Unit Division – Article describes how mixed unit division can be used to make the speed of animations proportional to screen size.
10. CSS Animation Fill Mode – Medium article describing a use case for fill-mode: backwards.
11. Pure CSS scrolling shadows with background-attachment: local – Article by Lea Verou describes using background images to implement container scroll indicators.
12. CSS Scroll Shadows – Article by Pedro Nave describes the scroll shadows design pattern. Also includes a Code Pen that uses mix-blend-mode to blend the background with the container contents.
13. Drawing Images with CSS Gradients – CSS Tricks article describes using various CSS gradients to compose a background image.
14. The --var: ; hack to toggle multiple values with one custom property – Lea Verou’s article describes the “space toggle” hack/trick that makes use of the CSS var() function’s fallback second parameter dependent on whether the desired parameter contains a space/empty character or an invalid string.
15. Should responsive images work with container queries? – Article by Stefan Judis describing multiple discussions on how container queries can be used to determine the size of images in responsive layouts.

©N. A. WEDDERBURN

16. 3 things about CSS variables you might not know – Article by Patrick Brosset describing CSS variable fallbacks and nesting.
17. CSS Flex overflow and center – Stack Overflow post describing how to add before and after pseudo elements to a non-wrapping row flexbox container that equally grow to fill the parent container to center the flexbox container in its parent.

Figures & Tables

Figure 1: Anchor with child image that has height greater than the line height. 13

Figure 2: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights. 13

Figure 3: Text anchor with inline-grid display not aligned with surrounding text. 14

Figure 4: Scrollable container with vertical overflow, scrolled to top of container. All scroll indicators except the bottom are covered, indicating user can scroll down. 16

Figure 5: Scrollable container with vertical overflow, scrolled to bottom of container. All scroll indicators except the top are covered, indicating user can scroll up. 17

Figure 6: Scrollable Container with both vertical and horizontal overflow. The vertical scrollbar hides the horizontal scroll indicator at the right edge of the container. The horizontal scrollbar hides the vertical scroll indicator at the bottom edge of the container. 18

Figure 7: Loader page loading animation timeline, reduced motion. 22

Figure 8: Loader exit animation timeline, reduced motion. 23

Figure 9: Loader page loading animation timeline, no motion preference. 24

Figure 10: Loader exit animation timeline, no motion preference. 25

Figure 11: Slide component. 28

Figure 12: Slide Component structure 29

Figure 13: Decorative header image for slides in flow layout. 30

Figure 14: Decorative background image for slides in fixed dimension layout. 31

Figure 15: The image on the left shows the navigation container with its intrinsic height partially obscuring the slide header. The image on the right shows the navigation container with enlarged top padding that does not obscure the slide header. 33

©N. A. WEDDERBURN

Figure 16: The image on the left shows the navigation container with enlarged padding creating additional space in the slide container. The image on the right shows the negative bottom margin on the navigation container's previous sibling element cancelling out the navigation's enlarged padding. . 33

Figure 17: Slideshow Presentation layout. 34

Figure 18: The image on the left shows a red decorative circle overflowing into its slide's bottom margin. The image on the right is the next slide displaying the overflowing decorative circle of the previous slide. 35

Figure 19: Firefox print preview with 100vh slide overflowing bottom page margin. 36

Figure 20: Slide Preview Presentation layout. 36

Figure 21: Full Document Slideshow Presentation first slide display 37

Figure 22: Full Document Slideshow Presentation last slide display 38

Figure 23: Indulgent Hydration Intro Slide animation timeline, no motion preference. 40

Figure 24: Indulgent Hydration New Arrivals Slide centered product list with before/after spacer pseudo elements 42

Figure 25: Indulgent Hydration New Arrivals Slide scrolling product list. The image on the left shows column gaps separating the flex items, including the pseudo element items, creating a gap at the start of the list. The image on the right shows right margins used to separate the flex items, removing the extra gap at the start of the list. 42

Figure 26: Call-to-action anchor with distorted shape from extra-large border radius hack 45

Table 1: Scrollable Container background image gradients and their purposes 15

Table 2: Product Component grid layout. 27

Table 3: Indulgent Hydration New Arrivals Slide, 1 column flexbox layout 42

Table 4: Indulgent Hydration New Arrivals Slide, element flex display properties 43

Table 5: Indulgent Hydration Collection Information Slide, 1 column flexbox layout in small viewport ... 43

Table 6: Indulgent Hydration Collection Information Slide, element flex display properties in small viewport 44

Table 7: Indulgent Hydration Collection Information Slide, element grid display properties in large viewport 44

©N. A. WEDDERBURN